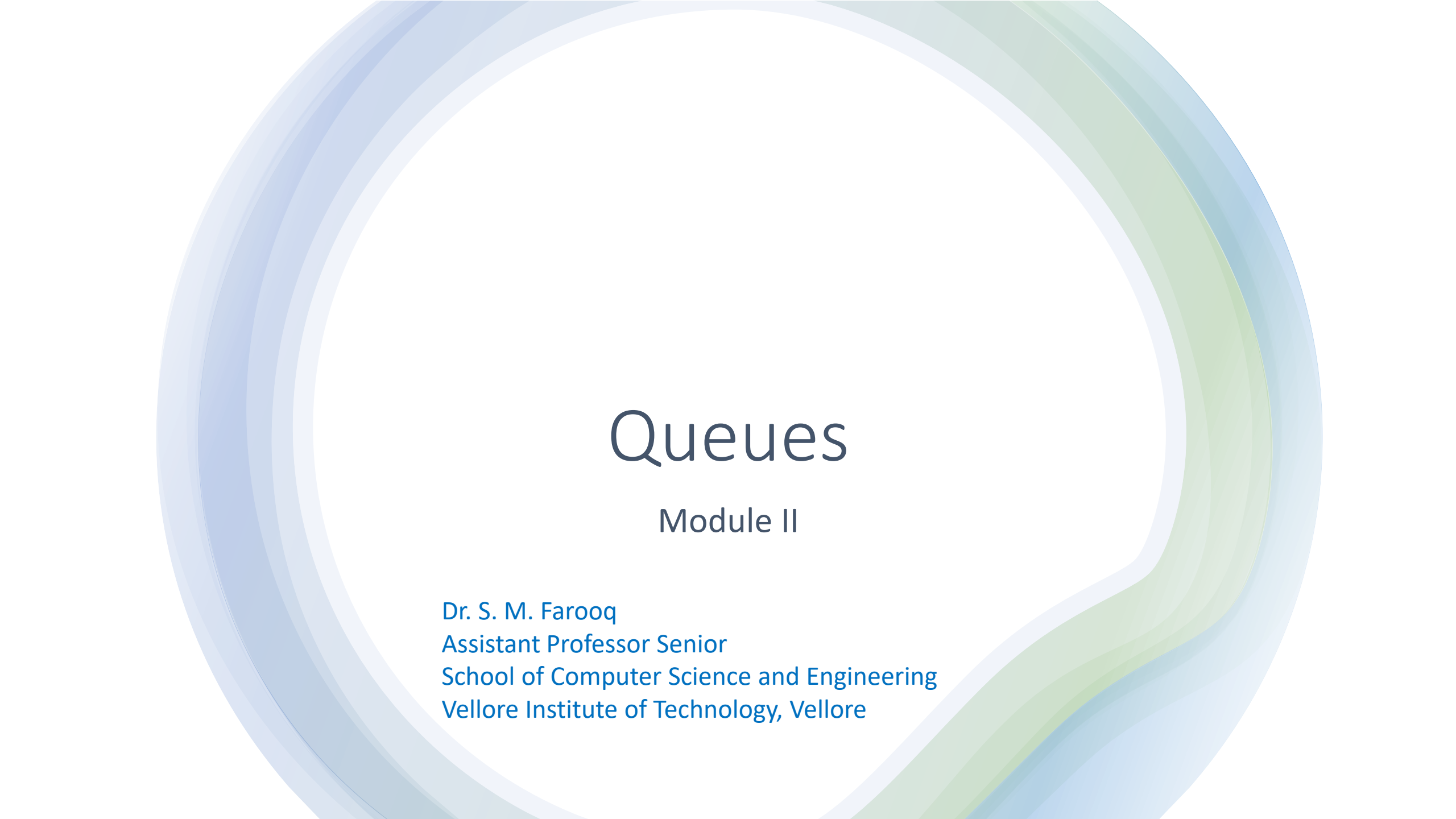




Queues

Module II

Dr. S. M. Farooq
Assistant Professor Senior
School of Computer Science and Engineering
Vellore Institute of Technology, Vellore

Introduction to Queues

- A queue is an ordered collection of items where the addition of new items happens at one end, called the “rear,” and the removal of existing items occurs at the other end, commonly called the “front.”
- Queue: FIFO (First In First Out) : Enqueue and Dequeue, front, rear
- Stack: FILO or LIFO , Push and Pop, top



0

1

2

3

4

5



front

0

rear

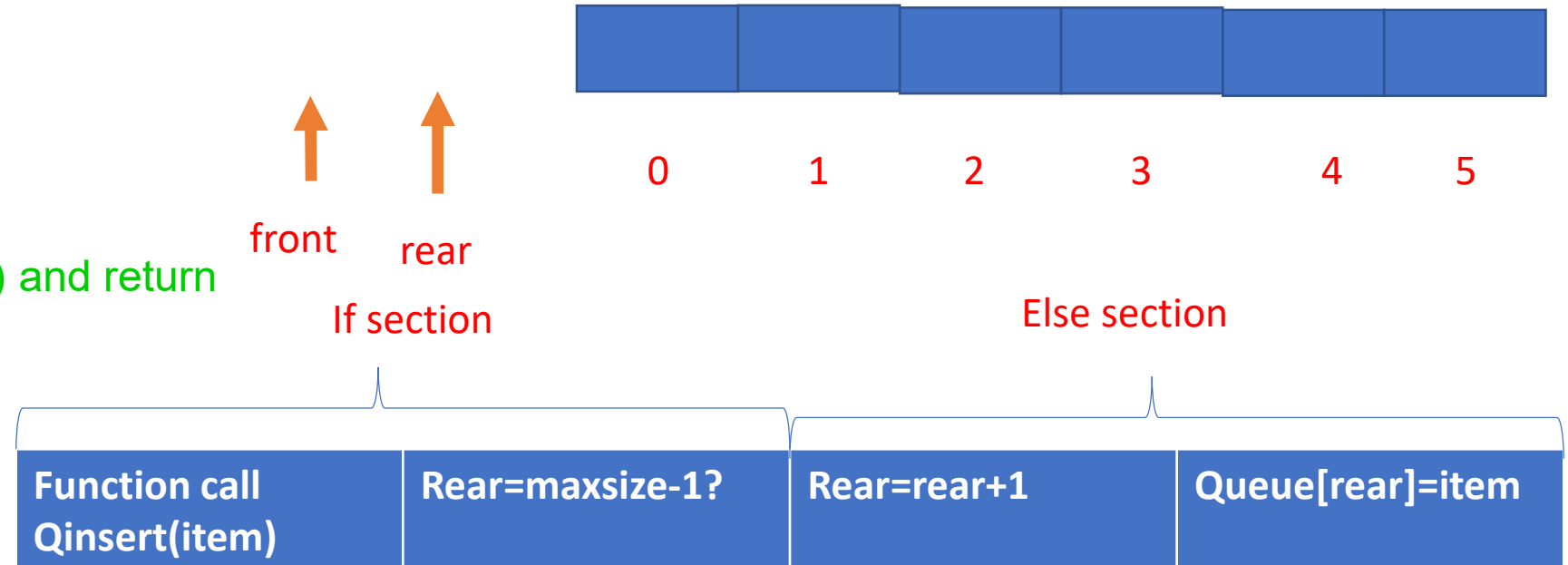
4

Array
representation
of Queue

There are two variables i.e.
front and rear, that are
implemented in the case of
every queue.

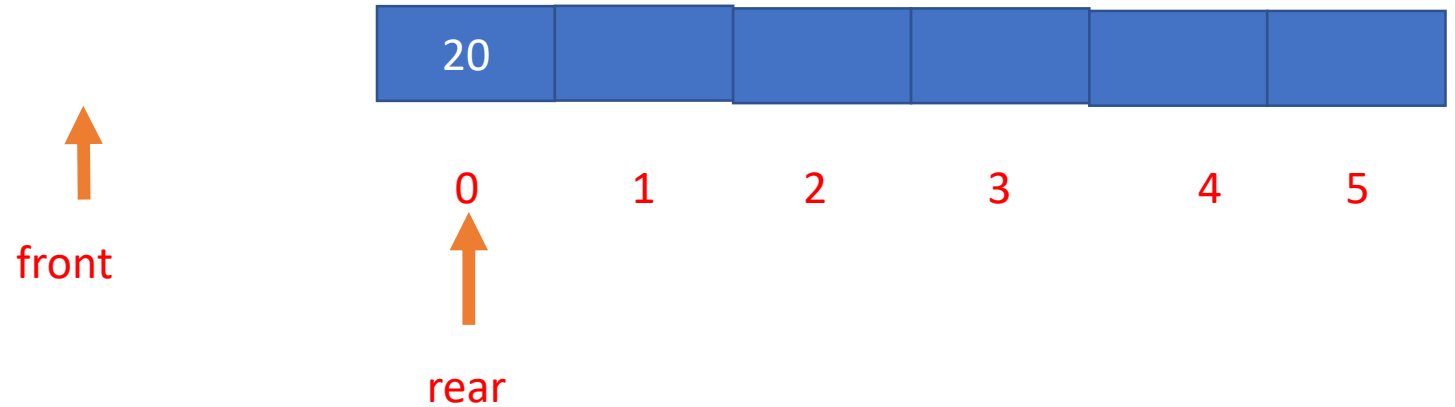
Maxsize=6, rear=front=-1

- *Qinsert(item)*
- 1.If (rear = maxsize-1)
- print ("queue overflow") and return
- 2.Else
- rear = rear + 1
- Queue [rear] = item



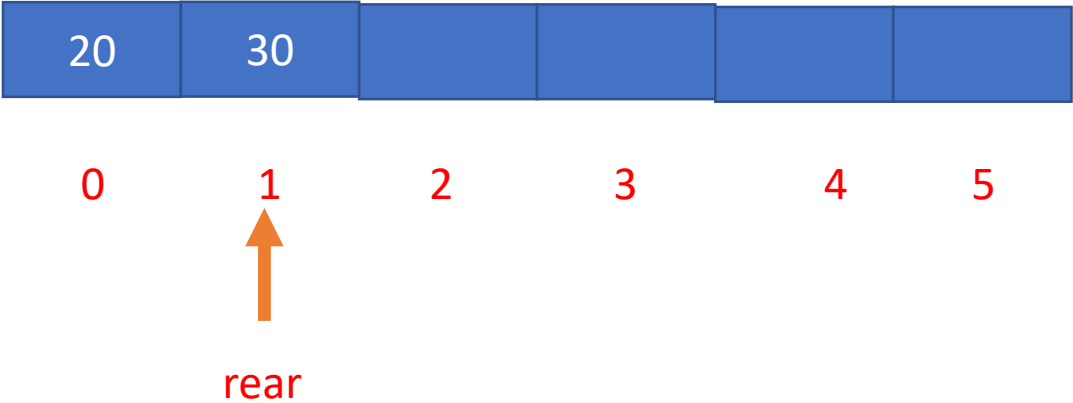
Maxsize=6, rear=0, front=-1

- *Qinsert(item)*
- 1.If (rear = maxsize-1)
- print ("queue overflow") and return
- 2.Else
- rear = rear + 1
- • Queue [rear] = item



If section		Else section	
Function call Qinsert(item)	Rear=maxsize-1	Rear=rear+1	Queue[rear]=item
1 Qinsert(20)	-1=5 ? F	Rear=-1+1=0	Queue[0]=20

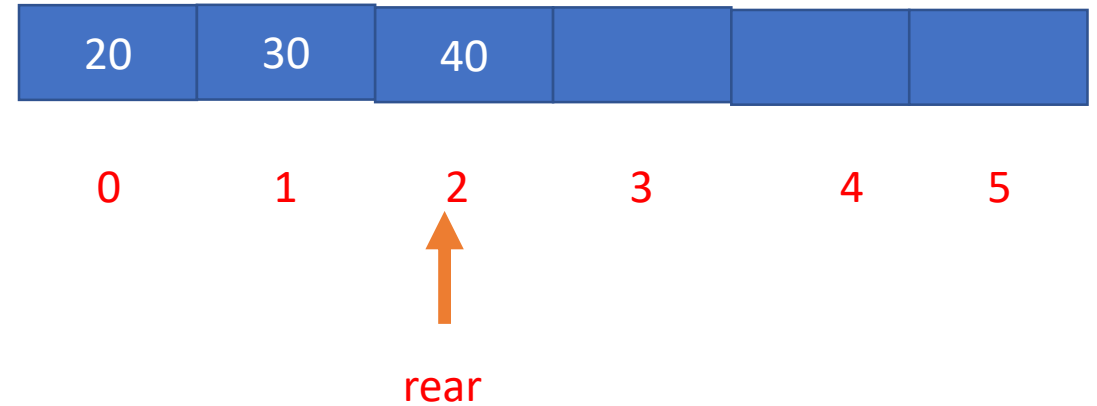
Maxsize=6, rear=1, front=-1



- *Qinsert(item)*
- 1.If (rear = maxsize-1)
- print (“queue overflow”) and return
- 2.Else
- rear = rear + 1
- • Queue [rear] = item

If section		Else section	
Function call Qinsert(item)	Rear=maxsize-1	Rear=rear+1	Queue[rear]=item
1 Qinsert(20)	-1=5 ? F	Rear=-1+1=0	Queue[0]=20
2 Qinsert(30)	0=5? F	Rear= 0+1=1	Queue[1]=30

Maxsize=6, rear=2, front=-1



- *Qinsert(item)*
- 1.If (rear = maxsize-1)
- print ("queue overflow") and return
- 2.Else
- rear = rear + 1
- • Queue [rear] = item

	If section		Else section	
	Function call Qinsert(item)	Rear=maxsize-1	Rear=rear+1	Queue[rear]=item
1	Qinsert(20)	-1=5 ? F	Rear=-1+1=0	Queue[0]=20
2	Qinsert(30)	0=5? F	Rear= 0+1=1	Queue[1]=30
3	Qinsert(40)	1=5? F	Rear= 1+1=2	Queue[2]=40

Maxsize=6, rear=3, front=-1



0 1 2 3 4 5

front



rear

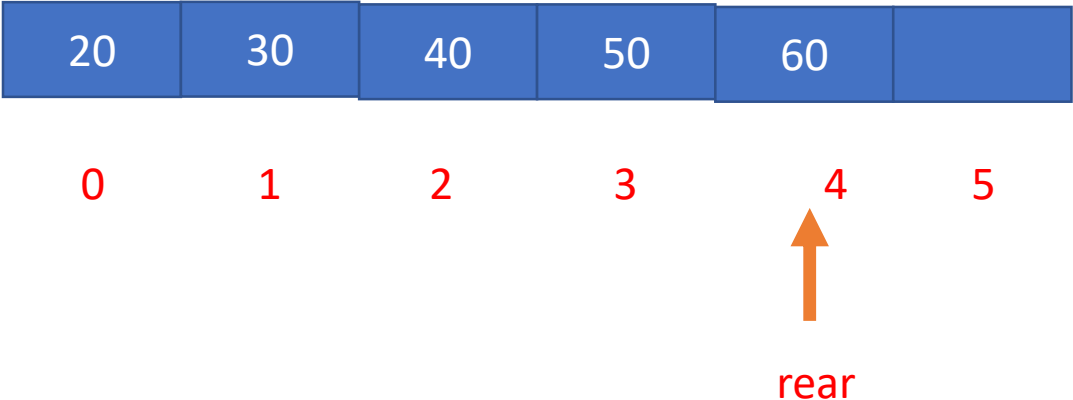
If section

Else section

→ • Queue [rear] = item

	Function call Qinsert(item)	Rear=maxsize-1	Rear=rear+1	Queue[rear]=item
1	Qinsert(20)	-1=5 ? F	Rear=-1+1=0	Queue[0]=20
2	Qinsert(30)	0=5? F	Rear= 0+1=1	Queue[1]=30
3	Qinsert(40)	1=5? F	Rear= 1+1=2	Queue[2]=40
4	Qinsert(50)	2=5? F	Rear= 2+1=3	Queue[3]=50

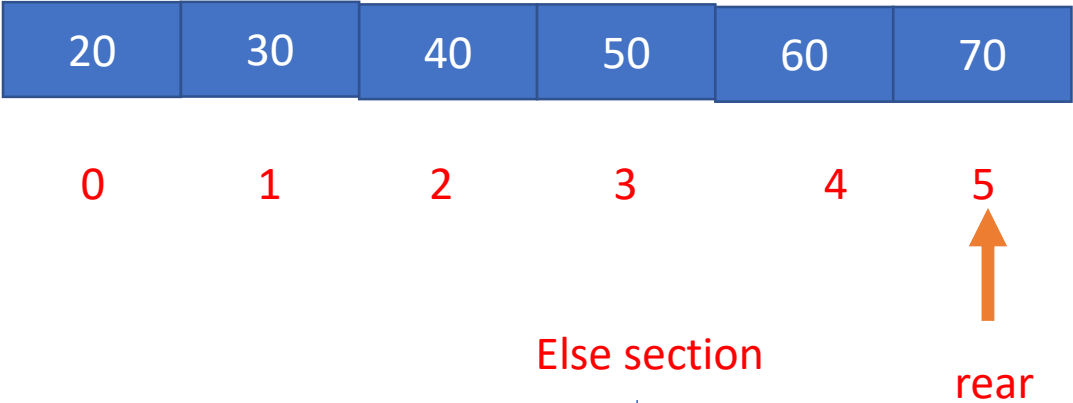
Maxsize=6, rear=4, front=-1



- *Qinsert(item)*
- 1.If (rear = maxsize-1)
- print (“queue overflow”) and return
- 2.Else
- rear = rear + 1
- • Queue [rear] = item

	If section		Else section	
	Function call Qinsert(item)	Rear=maxsize-1	Rear=rear+1	Queue[rear]=item
1	Qinsert(20)	-1=5 ? F	Rear=-1+1=0	Queue[0]=20
2	Qinsert(30)	0=5? F	Rear= 0+1=1	Queue[1]=30
3	Qinsert(40)	1=5? F	Rear= 1+1=2	Queue[2]=40
4	Qinsert(50)	2=5? F	Rear= 2+1=3	Queue[3]=50
5	Qinsert(60)	3=5? F	Rear= 3+1=4	Queue[4]=60

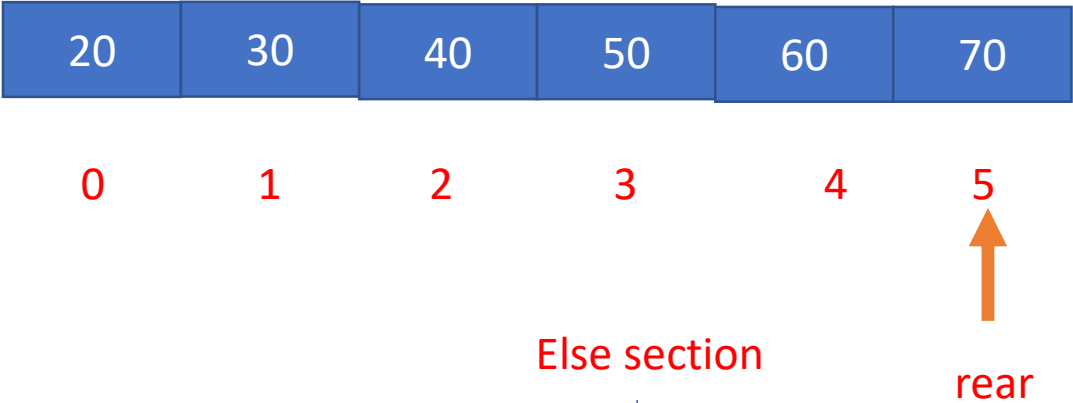
Maxsize=6, rear=5, front=-1



- *Qinsert(item)*
- 1.If (rear = maxsize-1)
- print (“queue overflow”) and return
- 2.Else
- rear = rear + 1
- • Queue [rear] = item

	Function call Qinsert(item)	Rear=maxsize-1	Rear=rear+1	Queue[rear]=item
1	Qinsert(20)	-1=5 ? F	Rear=-1+1=0	Queue[0]=20
2	Qinsert(30)	0=5? F	Rear= 0+1=1	Queue[1]=30
3	Qinsert(40)	1=5? F	Rear= 1+1=2	Queue[2]=40
4	Qinsert(50)	2=5? F	Rear= 2+1=3	Queue[3]=50
5	Qinsert(60)	3=5? F	Rear= 3+1=4	Queue[4]=60
6	Qinsert(70)	4=5? F	Rear= 4+1=5	Queue[5]=70

Maxsize=6, rear=5, front=-1

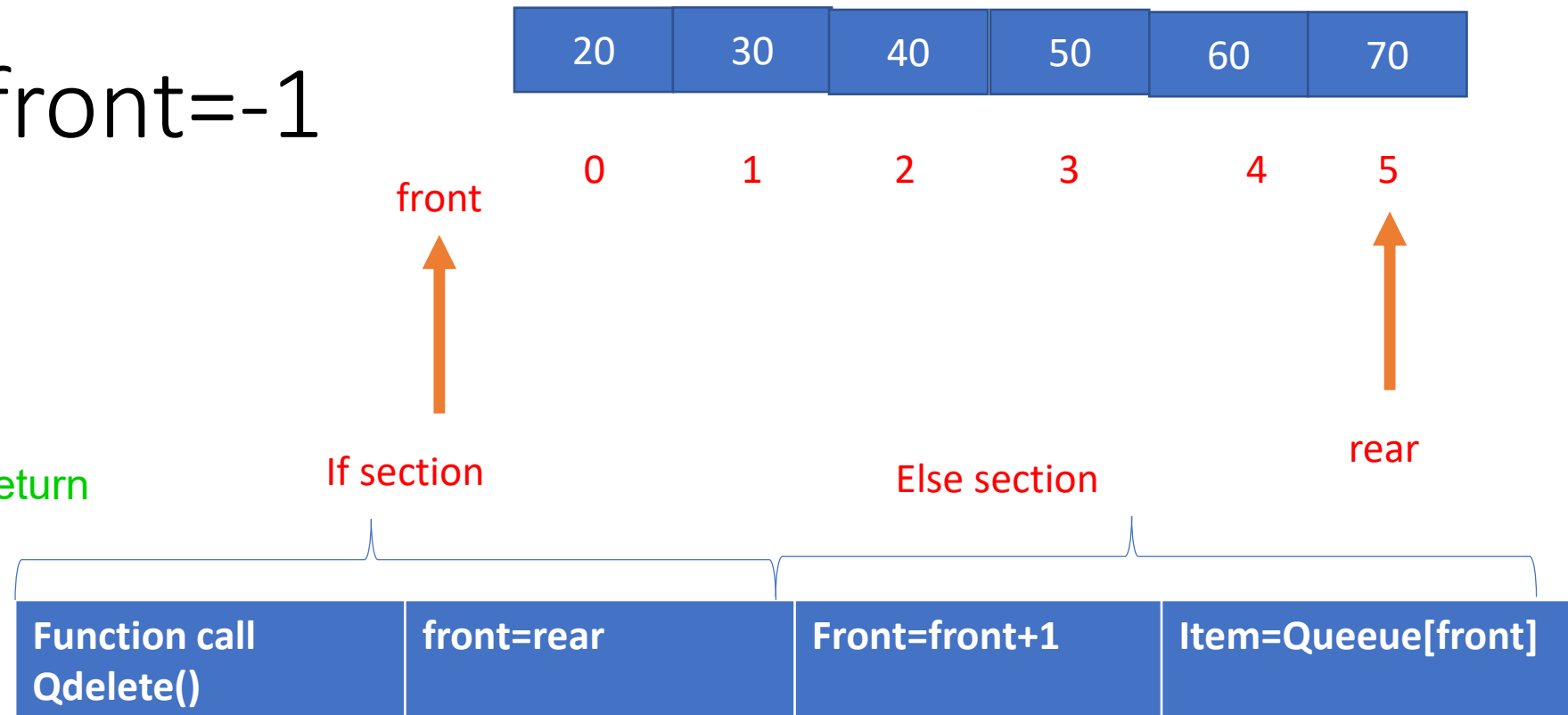


- *Qinsert(item)*
- 1.If (rear = maxsize-1)
- print (“queue overflow”) and return
- 2.Else
- rear = rear + 1
- • Queue [rear] = item

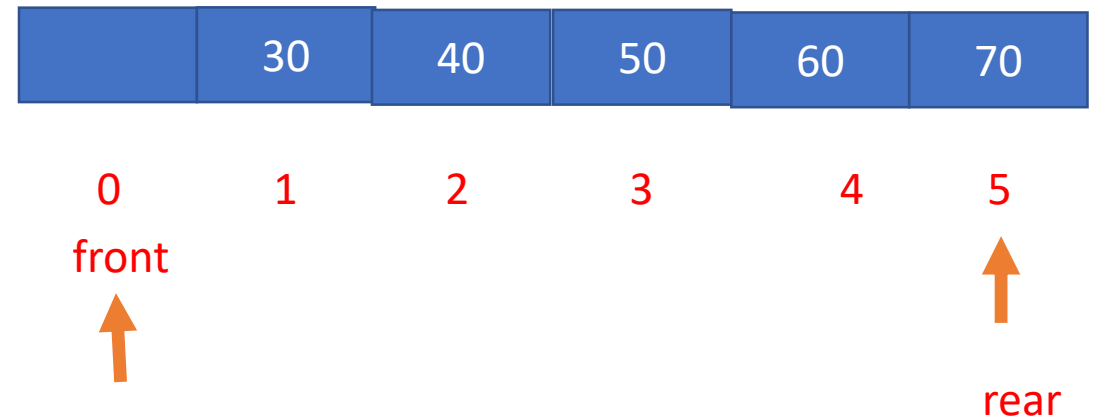
	Function call Qinsert(item)	Rear=maxsize-1	Rear=rear+1	Queue[rear]=item
1	Qinsert(20)	-1=5 ? F	Rear=-1+1=0	Queue[0]=20
2	Qinsert(30)	0=5? F	Rear= 0+1=1	Queue[1]=30
3	Qinsert(40)	1=5? F	Rear= 1+1=2	Queue[2]=40
4	Qinsert(50)	2=5? F	Rear= 2+1=3	Queue[3]=50
5	Qinsert(60)	3=5? F	Rear= 3+1=4	Queue[4]=60
6	Qinsert(70)	4=5? F	Rear= 4+1=5	Queue[5]=70
7	Qinsert(80)	5=5? T stop		

Maxsize=6, front=-1
rear=5

- *Qdelete()*
- If (front == rear)
- print "queue empty" and return
- 2. Else
- Front = front + 1
- item = queue [front];
- Return item



Maxsize=6, front=0
rear=5

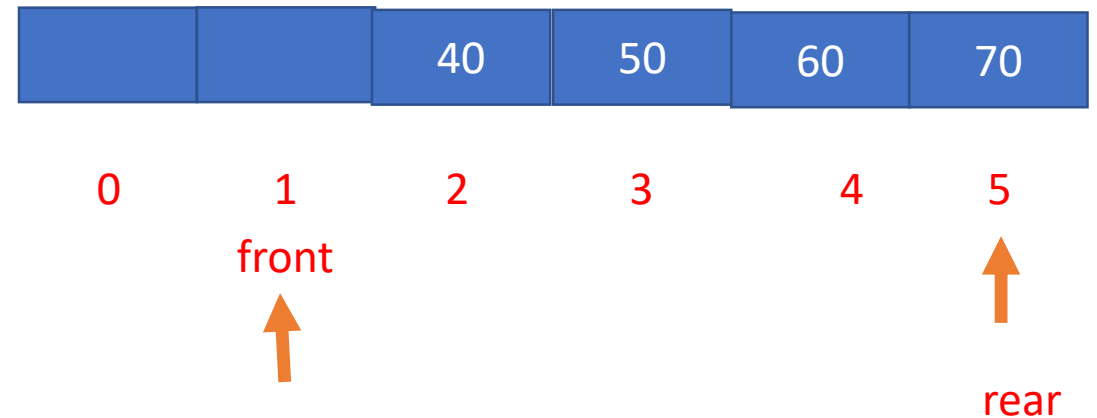


Item=20

- *Qdelete()*
- If (front =rear)
- print “queue empty” and return
- 2. Else
- Front = front + 1
- item = queue [front];
- Return item

If section		Else section	
Function call	front=rear	Front=front+1	Item=Queue[front]
1 Qdelete()	-1 = 5 ? F	front=-1+1=0	item=Queue[0]

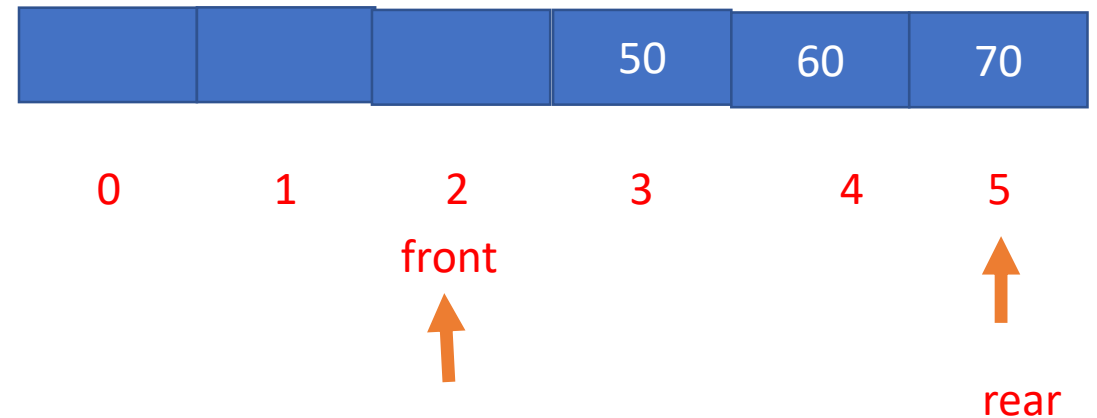
Maxsize=6, front=1
rear=5



- *Qdelete()*
- If (front =rear)
- print “queue empty” and return
- 2. Else
- Front = front + 1
- item = queue [front];
- Return item

If section		Else section	
Function call	front=rear	Front=front+1	Item=Queue[front]
1 Qdelete()	-1 = 5 ? F	front=-1+1=0	item=Queue[0]
2 Qdelete()	0 = 5 ? F	front=0+1=1	item=Queue[1]

Maxsize=6, front=2
rear=5

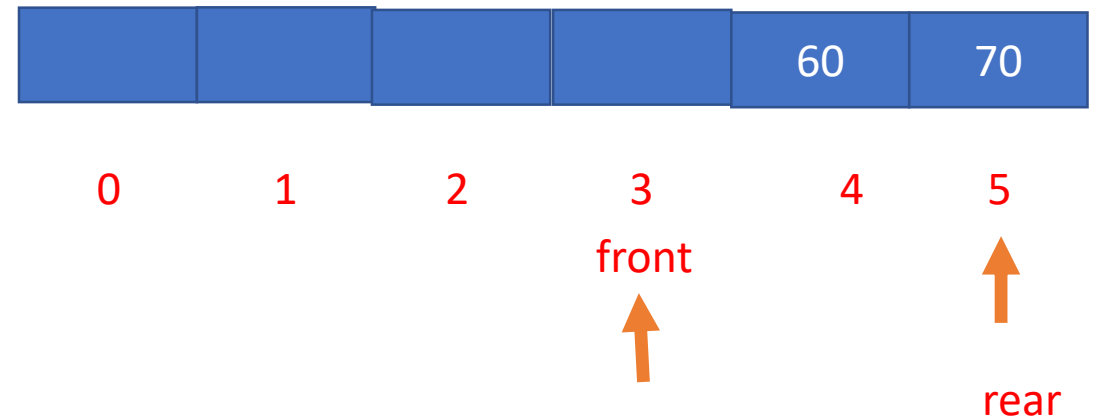


Item=40

- *Qdelete()*
- If (front =rear)
- print “queue empty” and return
- 2. Else
- Front = front + 1
- item = queue [front];
- Return item

	If section		Else section	
	Function call Qdelete()	front=rear	Front=front+1	Item=Queue[front]
1	Qdelete()	-1 = 5 ? F	front=-1+1=0	item=Queue[0]
2	Qdelete()	0 = 5 ? F	front=0+1=1	item=Queue[1]
3	Qdelete()	1 = 5 ? F	front=1+1=2	item=Queue[2]

Maxsize=6, front=3
rear=5

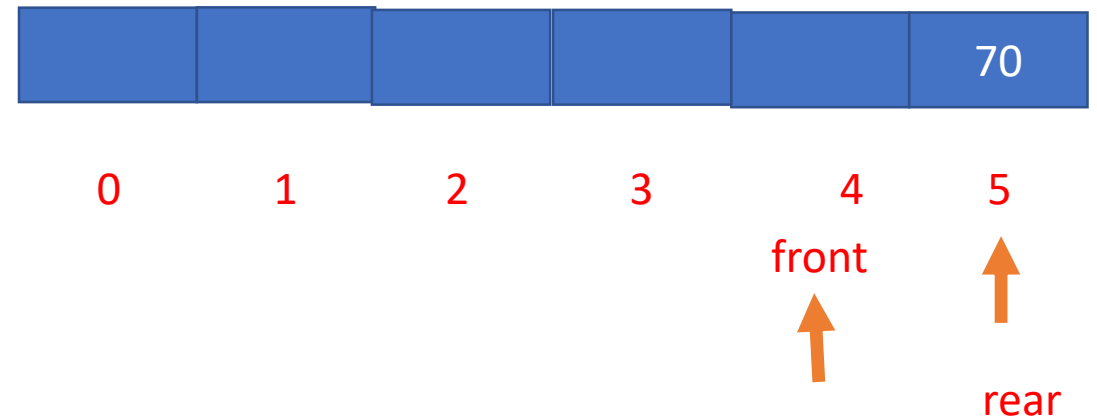


Item=50

- *Qdelete()*
- If (front =rear)
- print “queue empty” and return
- 2. Else
- Front = front + 1
- item = queue [front];
- Return item

	If section		Else section	
	Function call Qdelete()	front=rear	Front=front+1	Item=Queue[front]
1	Qdelete()	-1 = 5 ? F	front=-1+1=0	item=Queue[0]
2	Qdelete()	0 = 5 ? F	front=0+1=1	item=Queue[1]
3	Qdelete()	1 = 5 ? F	front=1+1=2	item=Queue[2]
4	Qdelete()	2 = 5 ? F	front=2+1=3	item=Queue[3]

Maxsize=6, front=4
rear=5

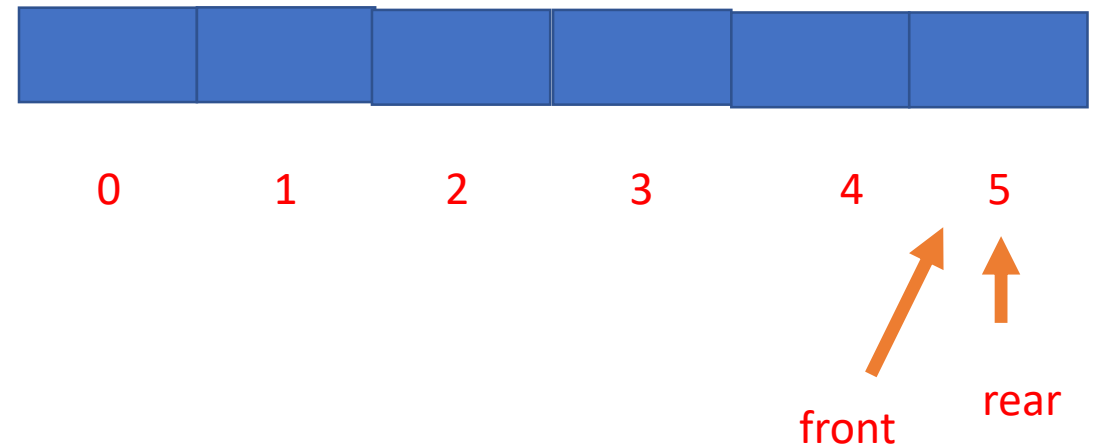


Item=60

- *Qdelete()*
- If (front =rear)
- print “queue empty” and return
- 2. Else
- Front = front + 1
- item = queue [front];
- Return item

	If section		Else section	
	Function call Qdelete()	front=rear	Front=front+1	Item=Queue[front]
1	Qdelete()	-1 = 5 ? F	front=-1+1=0	item=Queue[0]
2	Qdelete()	0 = 5 ? F	front=0+1=1	item=Queue[1]
3	Qdelete()	1 = 5 ? F	front=1+1=2	item=Queue[2]
4	Qdelete()	2 = 5 ? F	front=2+1=3	item=Queue[3]
5	Qdelete()	3 = 5 ? F	front=3+1=4	item=Queue[4]

Maxsize=6, front=5
rear=5



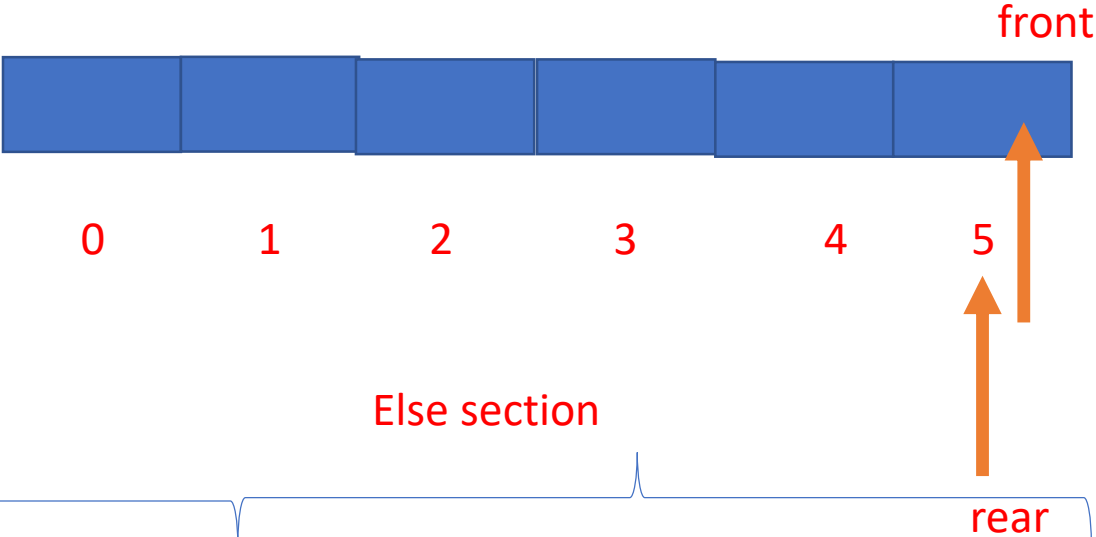
Item=70

- *Qdelete()*
- If (front =rear)
- print “queue empty” and return
- 2. Else
- Front = front + 1
- item = queue [front];
- Return item

	If section		Else section	
	Function call Qdelete()	front=rear	Front=front+1	Item=Queue[front]
1	Qdelete()	-1 = 5 ? F	front=-1+1=0	item=Queue[0]
2	Qdelete()	0 = 5 ? F	front=0+1=1	item=Queue[1]
3	Qdelete()	1 = 5 ? F	front=1+1=2	item=Queue[2]
4	Qdelete()	2 = 5 ? F	front=2+1=3	item=Queue[3]
5	Qdelete()	3 = 5 ? F	front=3+1=4	item=Queue[4]
6	Qdelete()	4 = 5 ? F	front=4+1=5	item=Queue[5]

Maxsize=6, front=5
rear=5

- *Qdelete()*
- If (front ==rear)
- print “queue empty” and return
- 2. Else
- Front = front + 1
- item = queue [front];
- Return item



	Function call Qdelete()	front=rear	Front=front+1	Item=Queue[front]
1	Qdelete()	-1 = 5 ? F	front=-1+1=0	item=Queue[0]
2	Qdelete()	0 = 5 ? F	front=0+1=1	item=Queue[1]
	Qdelete()	1 = 5 ? F	front=1+1=2	item=Queue[2]
	Qdelete()	2 = 5 ? F	front=2+1=3	item=Queue[3]
	Qdelete()	3 = 5 ? F	front=3+1=4	item=Queue[4]
6	Qdelete()	4= 5 ? F	front=4+1=5	item=Queue[5]
7	Qdelete()	5=5? T queue empty	-	-

Conditions to be satisfied in Queue

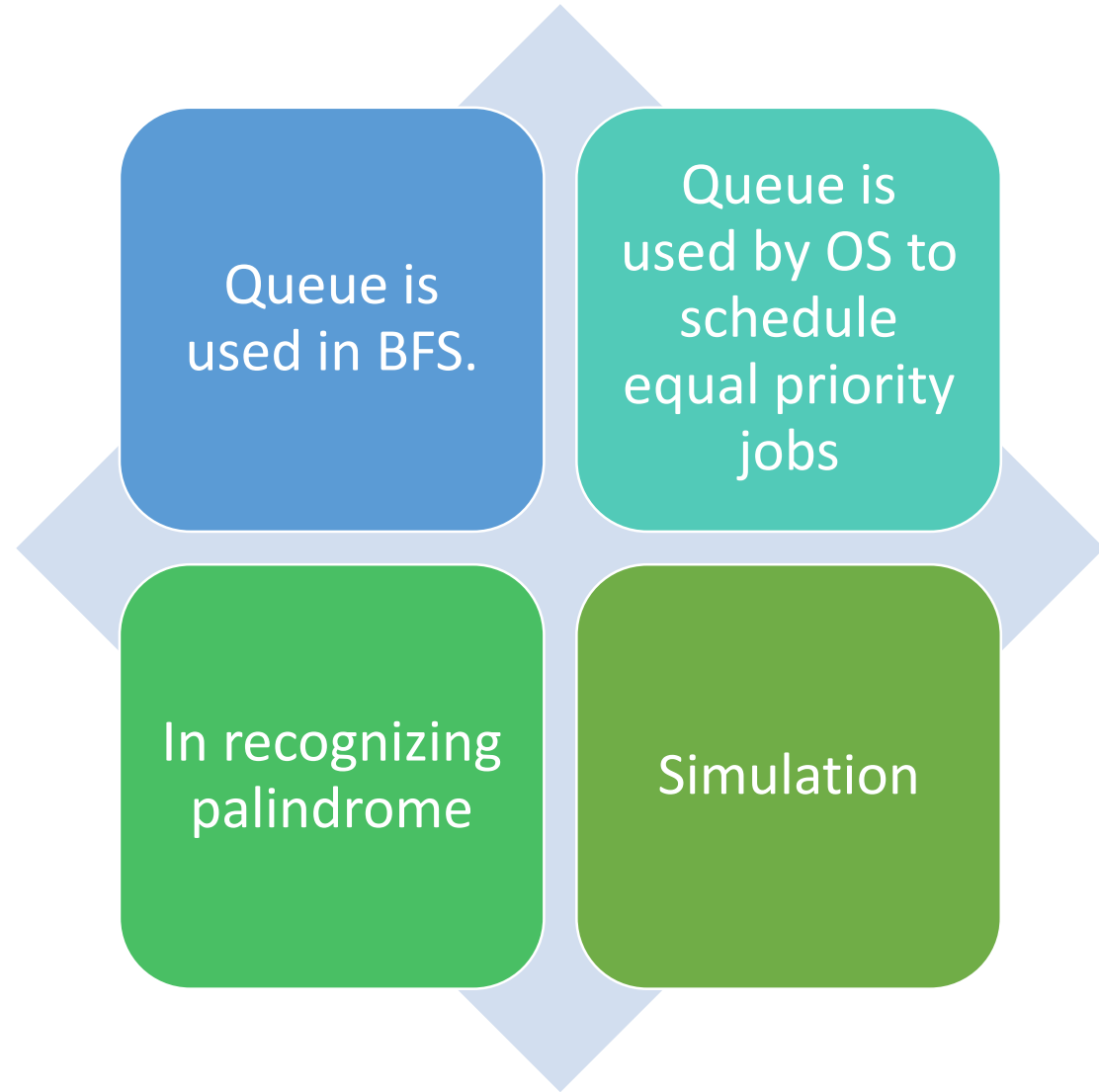
- Queue is empty $\text{FRONT} = \text{REAR}$
- Queue is full $\text{REAR} = \text{Maxsize}$
- Queue contains element ≥ 1

$$\text{FRONT} < \text{REAR}$$

$$\text{NO. OF ELEMENT} = \text{REAR} - \text{FRONT} + 1$$

Disadvantage of Queue: The deleted element's space in the queue cannot be reused-utilized for inserting another element.

Applications of queues





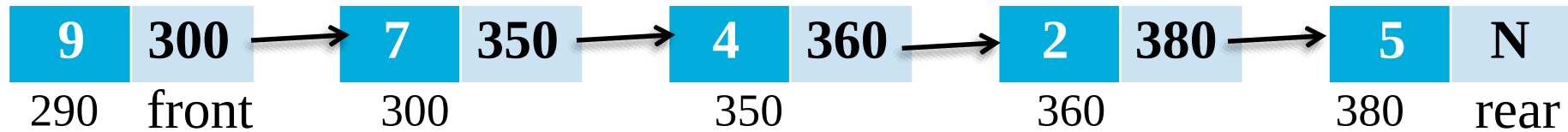
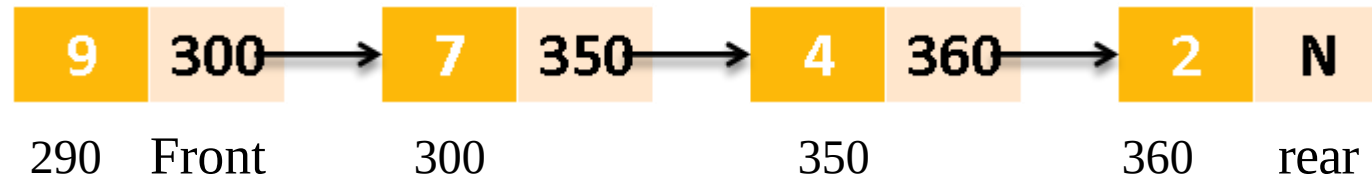
Representation of Queues

- Queues can be represented using arrays
- Queues can be represented using Linked List

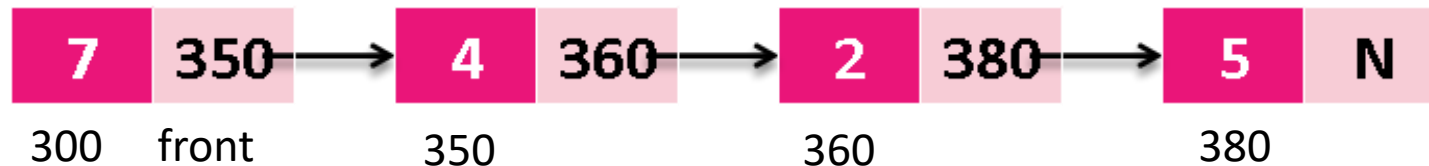
Array representation of Queues



Linked representation of Queues



Linked queue after inserting a node



Linked queue after deleting a node

FRONT=REAR=NULL

Insert an element in queue using linked list

STEP-1: Allocate memory for the new node & name it as TEMP.

STEP-2: SET TEMP → DATA = NUM

SET TEMP → LINK = NULL

STEP-3: IF FRONT = NULL

FRONT=REAR=TEMP

ELSE

REAR → LINK = TEMP

REAR=TEMP

[END OF IF]

STEP-4: EXIT

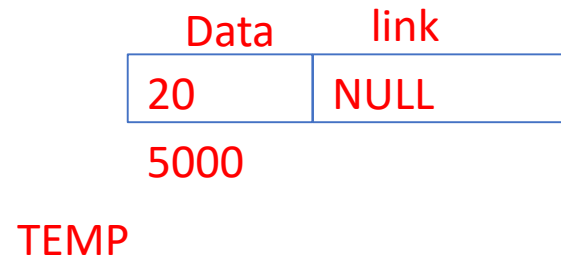
1. QINSERT_LINK(TEMP)

STEP-1: Allocate memory for the new node & name it as TEMP.

STEP-2: SET TEMP → DATA = NUM

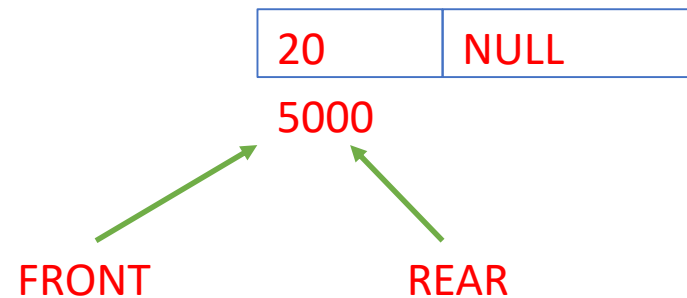
SET TEMP → LINK = NULL

FRONT=REAR=NULL



```
STEP-3:  IF FRONT = NULL
          FRONT=REAR=TEMP
        ELSE
          REAR → LINK =TEMP
          REAR=TEMP
        [ END OF IF]
```

FRONT=REAR=5000



2. QINSERT_LINK(TEMP)

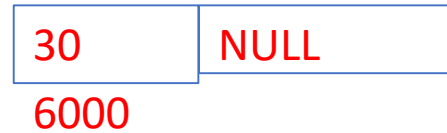
STEP-1: Allocate memory for the new node & name it as TEMP.

STEP-2: SET TEMP → DATA = NUM

SET TEMP → LINK = NULL

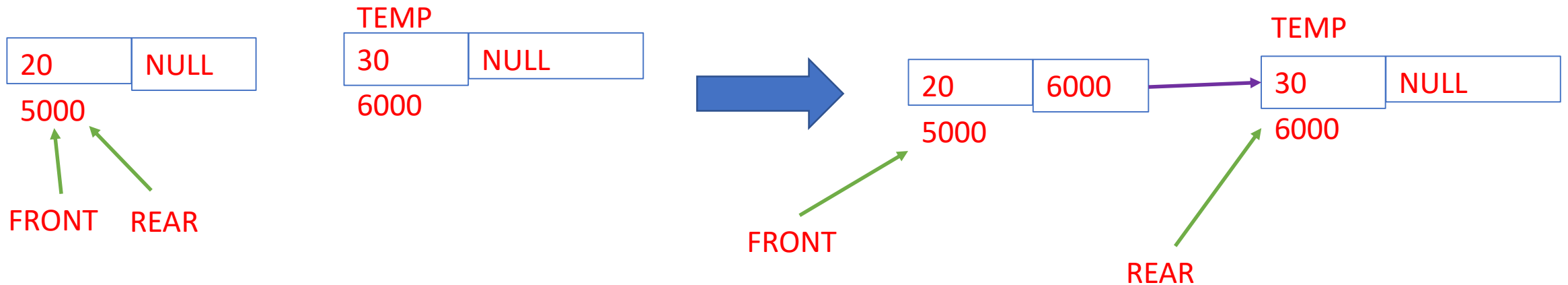
FRONT=REAR=TEMP

TEMP



STEP-3: IF FRONT = NULL
FRONT=REAR=TEMP
ELSE
REAR → LINK = TEMP
REAR=TEMP
[END OF IF]

FRONT=5000
REAR=6000



3. QINSERT_LINK(TEMP)

STEP-1: Allocate memory for the new node & name it as TEMP.

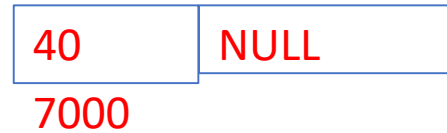
STEP-2: SET TEMP → DATA = NUM

SET TEMP → LINK = NULL

FRONT=5000

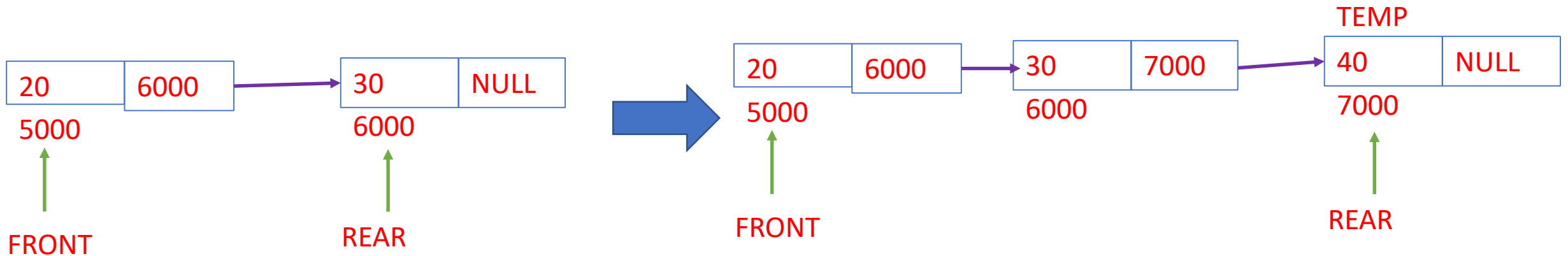
REAR=6000

TEMP

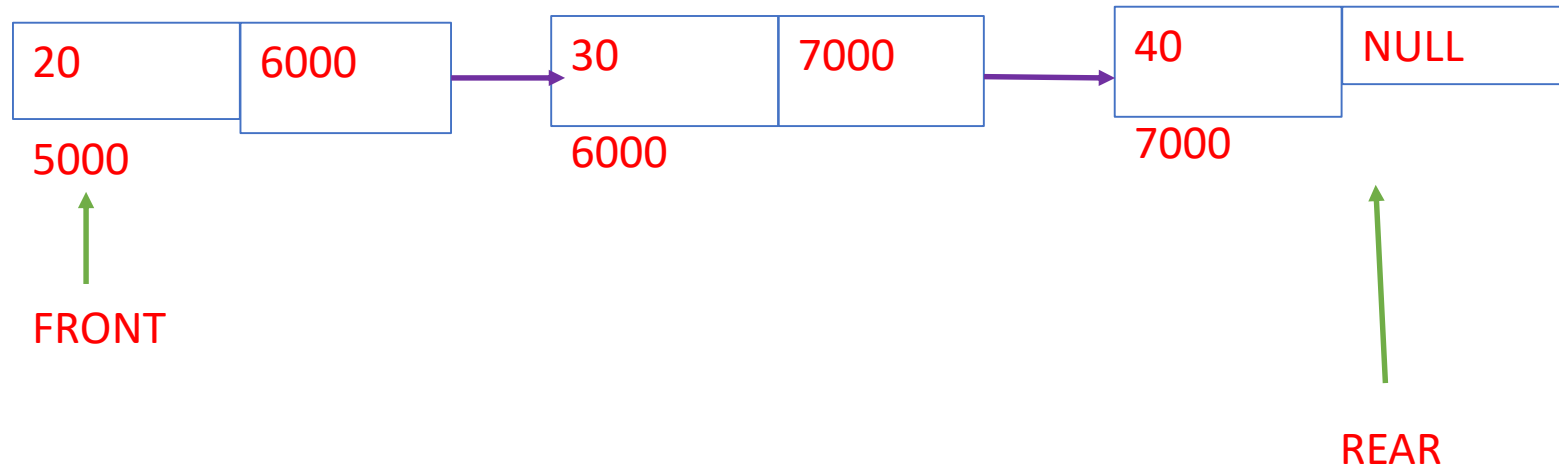


STEP-3: IF FRONT = NULL
FRONT=REAR=TEMP
ELSE
REAR → LINK = TEMP
REAR=TEMP
[END OF IF]

FRONT=5000
REAR=7000



FRONT=5000
REAR=7000



Delete an element in queue using linked list

STEP-1: If FRONT = NULL

 write “underflow”

 go to step 3.

 [End of if]

STEP-2: SET TEMP = FRONT

 FRONT=FRONT→ LINK

 IF FRONT = NULL

 REAR=NULL

STEP-3: EXIT

Delete an element in queue using linked list

1. QDELETE_LINK()

STEP-1: If FRONT = NULL

write "underflow"

go to step 3.

[End of if]

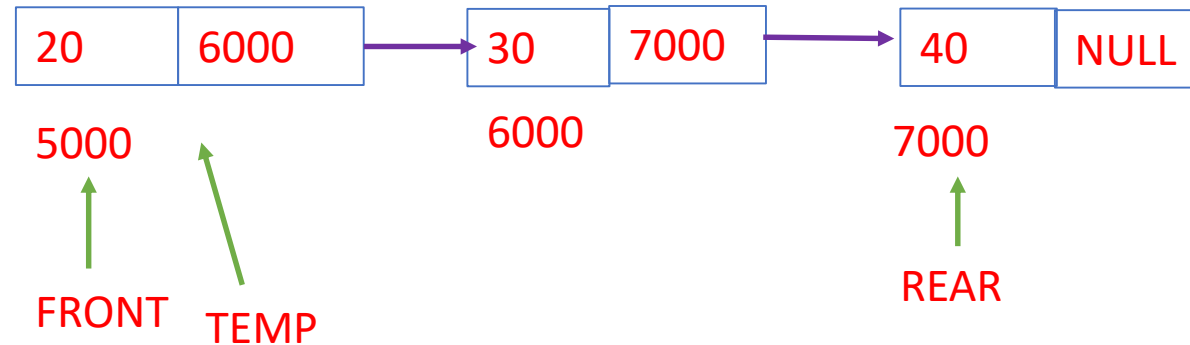
STEP-2: SET TEMP = FRONT

FRONT = FRONT → LINK

IF FRONT = NULL

REAR = NULL

STEP-3: EXIT



Delete an element in queue using linked list

1. QDELETE_LINK()

STEP-1: If FRONT = NULL
write "underflow"
go to step 4.

[End of if]

STEP-2: SET TEMP = FRONT

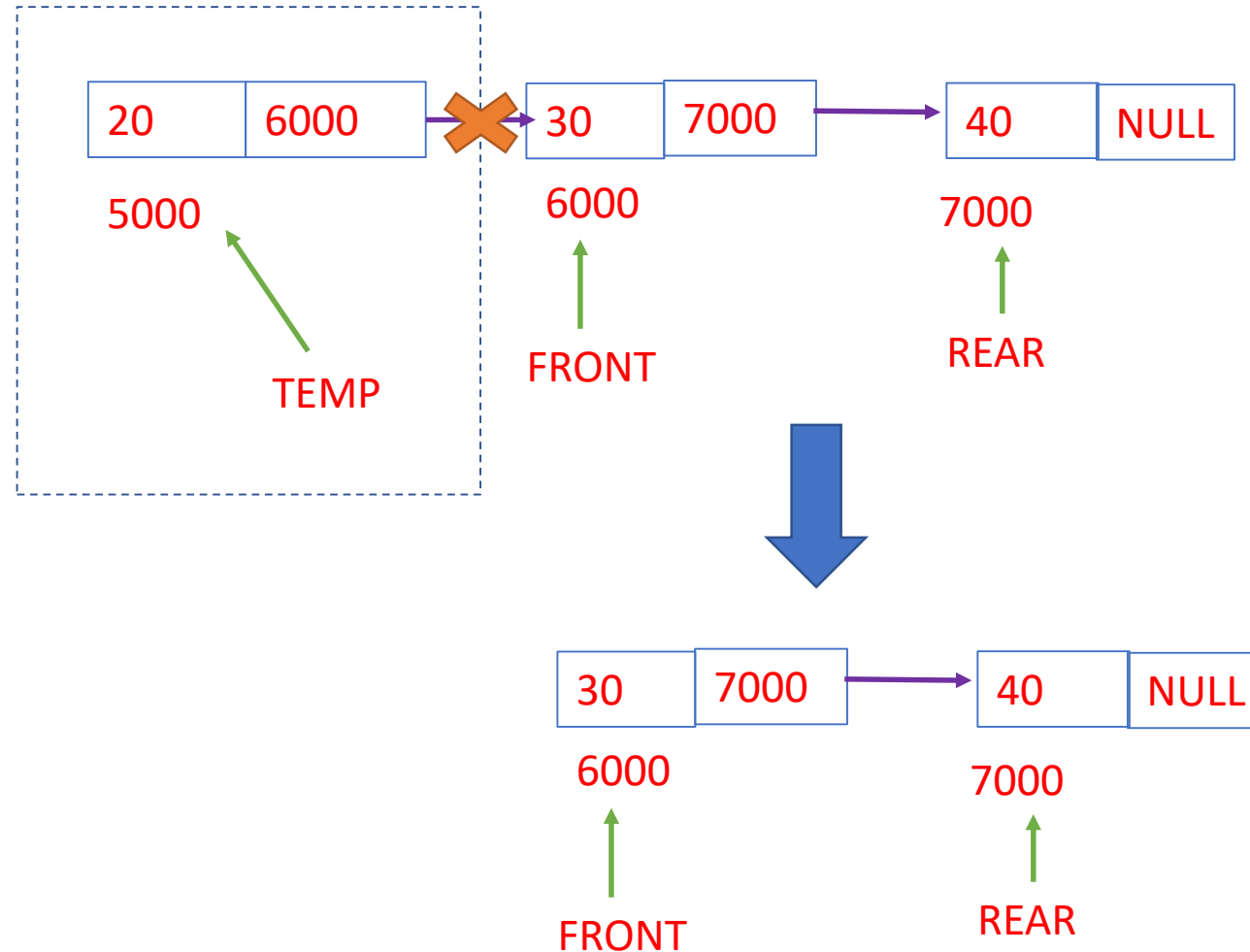
FRONT=FRONT→ LINK

IF FRONT = NULL

REAR=NULL

STEP-3: FREE(TEMP)

STEP-4: EXIT



2. QDELETE_LINK()

Delete an element in queue using linked list

STEP-1: If FRONT = NULL
write "underflow"
go to step 4.

[End of if]

STEP-2: SET TEMP = FRONT

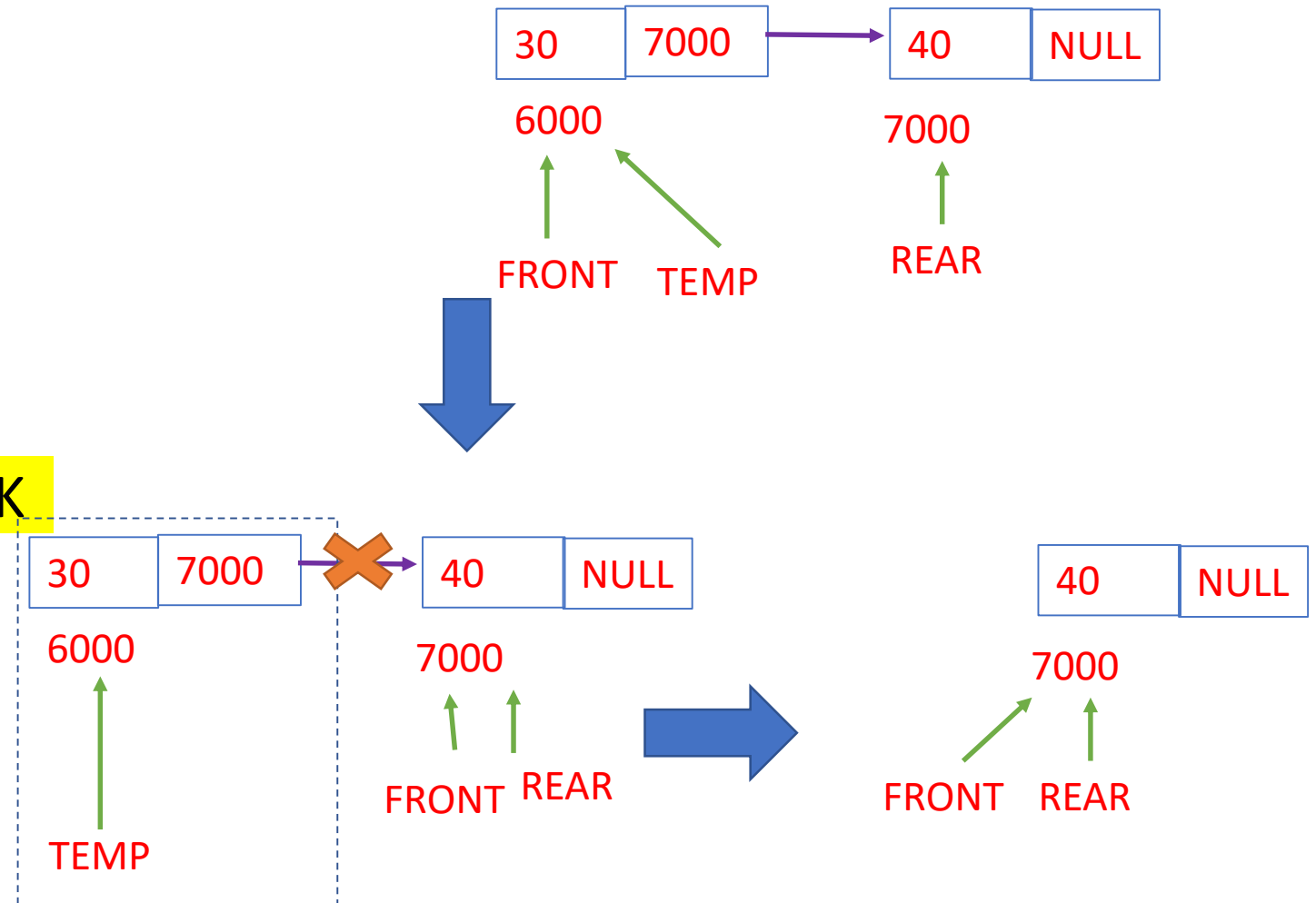
FRONT=FRONT→LINK

IF FRONT = NULL

REAR=NULL

STEP-3: FREE(TEMP)

STEP-4: EXIT



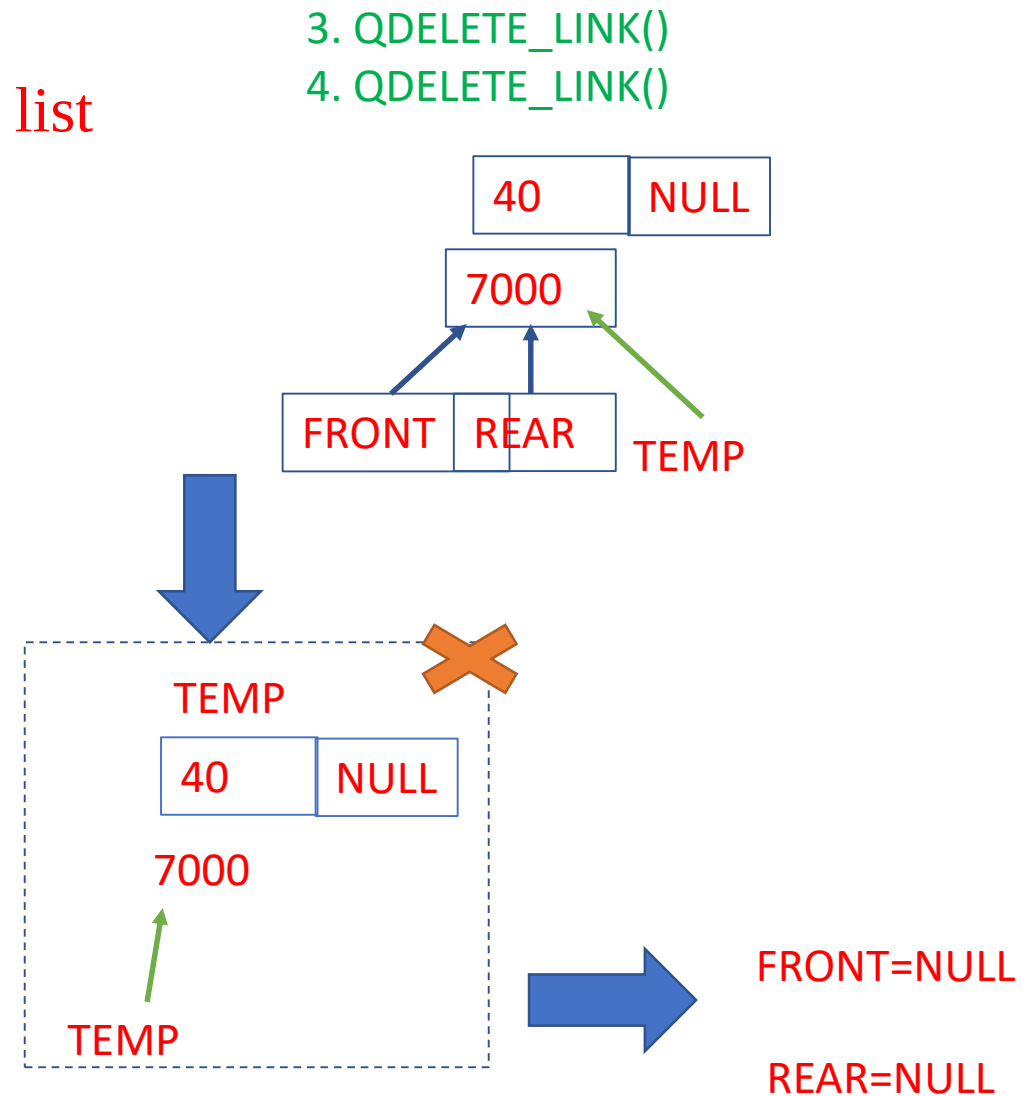
Delete an element in queue using linked list

STEP-1: If FRONT = NULL
 write “underflow”
 go to step 4.
 [End of if]

STEP-2: SET TEMP = FRONT
 FRONT=FRONT→ LINK
 IF FRONT = NULL
 REAR=NULL

STEP-3: FREE(TEMP)

STEP-4: EXIT

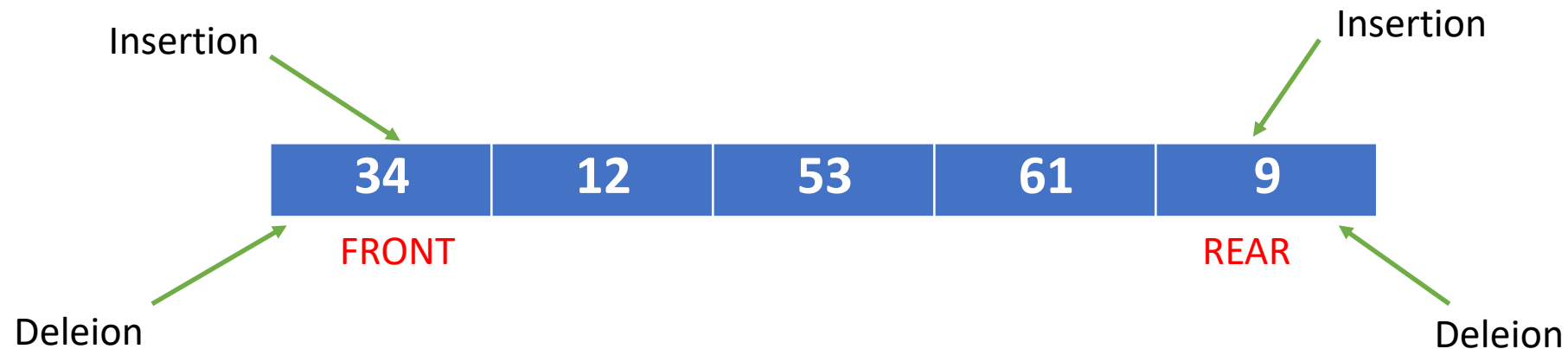


Types of Queues

- Dequeue
- Circular Queue
- Priority Queue

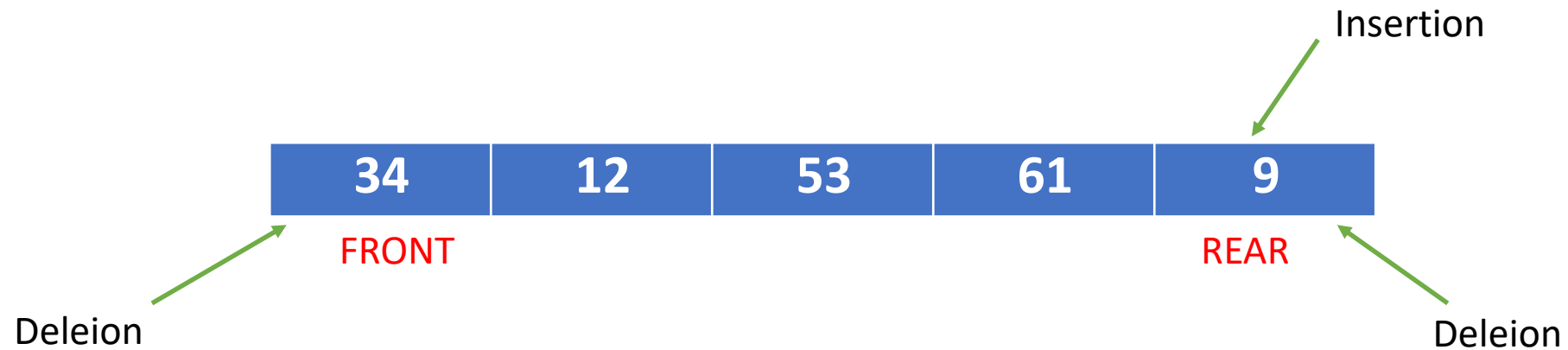
Dequeues

- Deque stands for double ended queue.
- Elements can be inserted or deleted at either end.
- Also known as head-tail linked list.



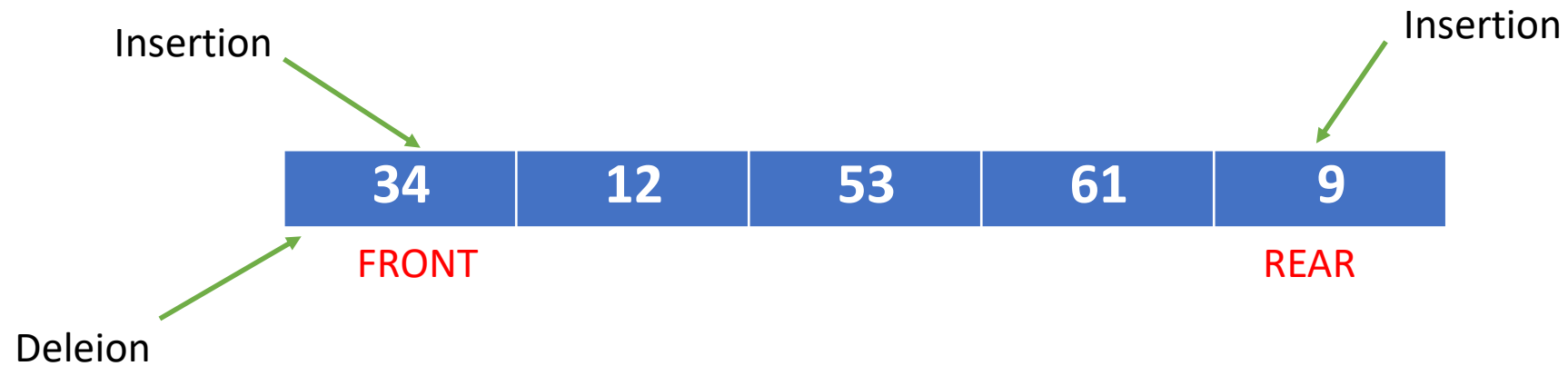
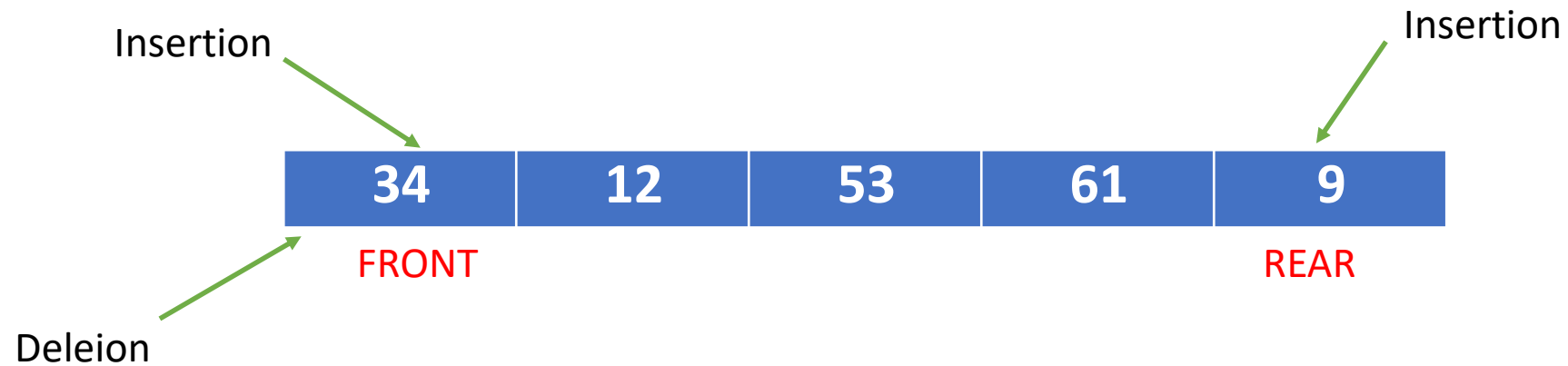
Types of Dequeues

- Input restricted queue



Types of Dequeues

- Output restricted queue



Circular Queues

- Circular queue are used to remove the drawback of simple queue.
- Both the front and the rear pointers **wrap around** to the beginning of the array.
- It is also called as “Ring buffer”.

Algorithm CQ_insert(10) modified

- Step-1: If $(\text{REAR}+1)\% \text{MAX} = \text{FRONT}$ [Check if Queue is Full]
 - Write Overflow
 - [End of if]
- Step-2: If $\text{REAR} = -1 \ \&\& \ \text{FRONT} = -1$ then [Check if Queue is Empty]
 - $\text{REAR} = \text{FRONT} = 0$
 - Else
 - $\text{REAR} = (\text{REAR}+1)\% \text{MAX}$
 - [End of If]
- Step-3: $\text{CQ}[\text{REAR}] = \text{ITEM}$
- Step-4: Exit

Insert 10, 20, 30, 40 ,50, 60 and 70 into CQ

Algorithm CQ_insert(10)

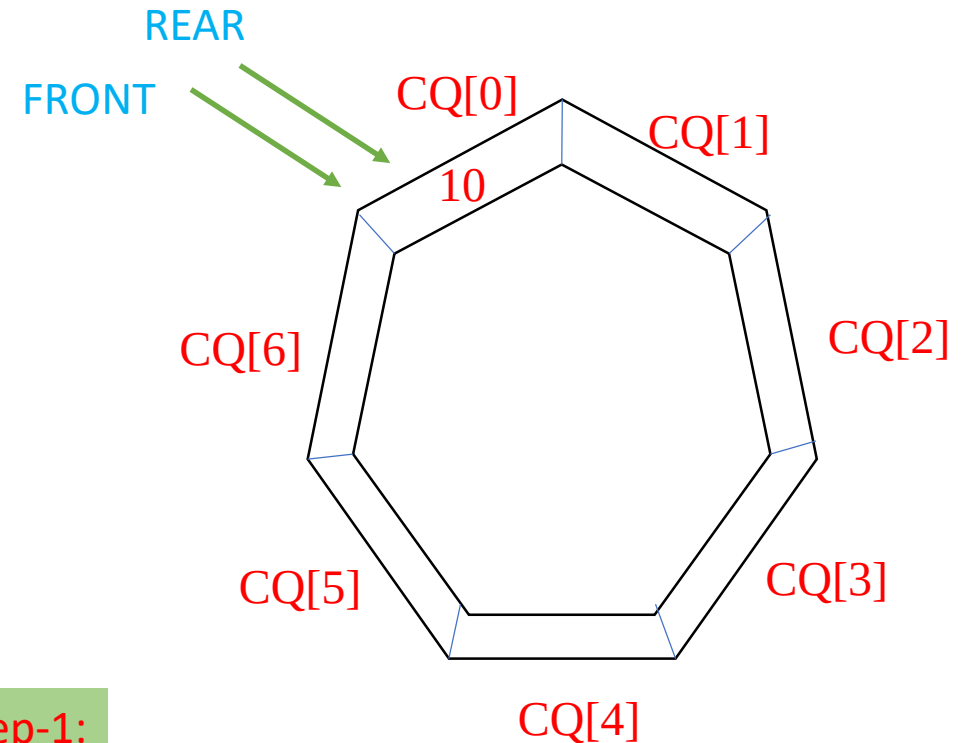
- Step-1: If $(\text{REAR}+1)\% \text{MAX} = \text{FRONT}$
- Write Overflow
- [End of if]
- Step-2: If $\text{REAR} = -1 \ \&\& \ \text{FRONT} = -1$ then
- $\text{REAR} = \text{FRONT} = 0$
- Else
- $\text{REAR} = (\text{REAR} + 1) \% \text{MAX}$
- [End of If]
- Step-3: $\text{CQ}[\text{REAR}] = \text{ITEM}$
- Step-4: Exit

Trace this algorithm for the remaining elements

FRONT=-1
REAR=-1
MAX=7



FRONT=0
REAR=0
MAX=7



Step-1:

IF $(\text{FRONT} = (\text{REAR} + 1) \% \text{MAX})$
 $-1 = (-1 + 1) \% 7$
 $-1 = 0$
 $-1 = 0$ False



Step-2: If part

IF $\text{FRONT} = -1 \ \&\& \ \text{REAR} = -1$? True
 $\text{REAR} = \text{FRONT} = 0$

Step-3:

$\text{CQ}[\text{REAR}] = 10$
 $\text{CQ}[0] = 10$

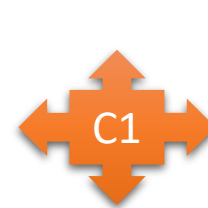
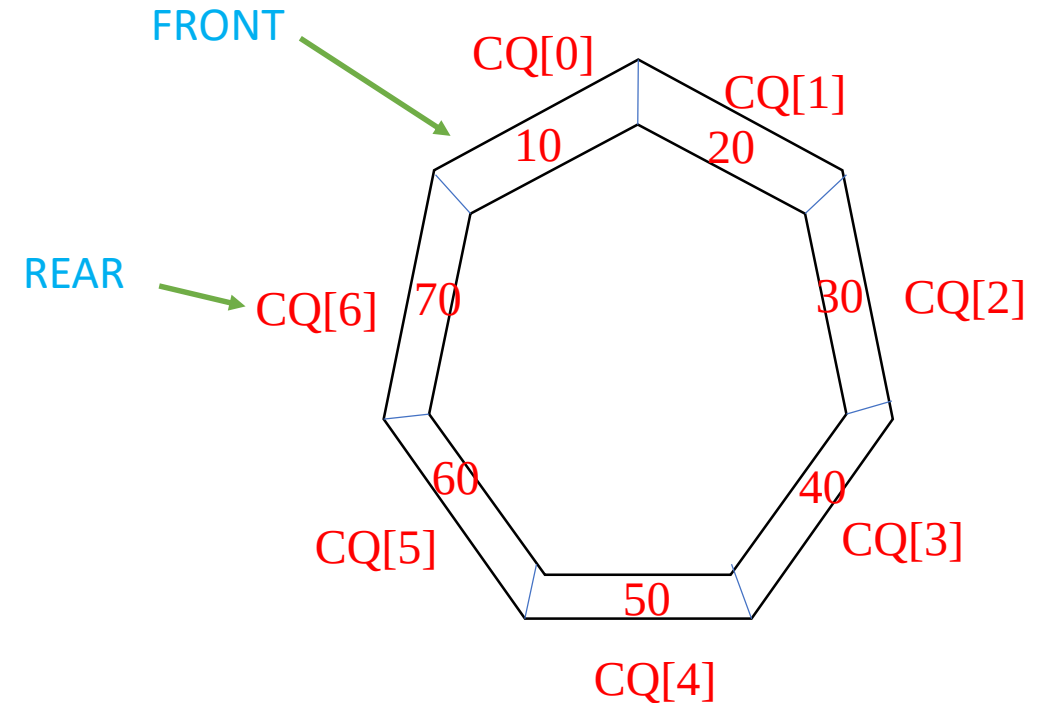
Algorithm CQ_delete()

- Step-1 : If $\text{FRONT} = -1 \ \&\& \ \text{REAR} = -1$ Then [Check if Queue is Empty]
 - Write Overflow
 - go to step 3
 - [End of if]
- Step-2: If $\text{FRONT} = \text{REAR}$ [If there is only one element in a Queue]
 - $\text{REAR} = \text{FRONT} = -1$
 - Else
 - $\text{FRONT} = (\text{FRONT} + 1) \% \text{MAX}$
 - [End of If]
- Step-3 : SET $\text{VAL} = \text{QUEUE}[\text{FRONT}]$ //Assuming as the element deleted
- Step-4: Exit

Algorithm CQ_delete()

- Step-1 : If FRONT=-1 && REAR = -1 Then
 - Write Overflow
 - go to step 3
 - [End of if]
- Step-2: If FRONT=REAR
 - REAR=FRONT=-1
 - Else
 - FRONT=(FRONT+1)%MAX
 - [End of If]
- Step-3 : SET VAL = QUEUE[FRONT]
- Step-4: Exit

FRONT=0
REAR=6
MAX=7



IF (FRONT=-1)
0=-1 F



IF (FRONT=REAR)
0=6 F
REAR=FRONT=-1 SKIP

ELSE
FRONT=(FRONT+1)%MAX
= (0+1)%7
= 1

PRIORITY QUEUE

- A priority Queue is a collection of elements where each element is assigned a priority and the order in which elements are deleted and processed is determined from the following rules:
- 1) An element of higher priority is processed before any element of lower priority.
- 2) Two elements with the same priority are processed according to the order in which they are added to the queue.

The priority queue implementation

- The priority queue is again implemented in two way
 1. array/sequential representation.
 2. Dynamic/ linked representation.

In priority queue node is divided into three parts



Structure of priority queue Node

PRIORITY QUEUE

- An example where priority queue are used is in operating systems.
- The operating system has to handle a large number of jobs.
- These jobs have to be properly scheduled.
- The operating system assigns priorities to each type of job.
- The jobs are placed in a queue and the job with the highest priority will be executed first.

PRIORITY QUEUE

- **Advantages:-**
 - Preferences to the higher priority process are added at the beginning.
 - Keep the list sorted in *increasing* order.

Applications of Queue

- Queue, as the name suggests is used whenever we need to have any group of objects in an order in which the first one coming in, also gets out first while the others wait for their turn, like in the following scenarios :
 - Serving requests on a single shared resource, like a printer, CPU task scheduling etc.
 - In real life, Call Center phone systems will use Queues, to hold people calling them in an order, until a service representative is free.
 - Handling of interrupts in real-time systems. The interrupts are handled in the same order as they arrive, First come first served.